

ARQUITETURA ORIENTADA A OBJETOS

Surgiu em 1970 e se popularizou em 1980.

Utiliza princípios de programação orientada a objetos:

- Objetos, classes, encapsulamento, abstração, herança e polimorfismo.

Nesse sentido:

- Enfatiza a interação entre objetos;
- Código é estruturado em torno de entidades do mundo real; e
- Comumente usados em Java e C++.

O objetivo dessa arquitetura é utilizar um dos principais paradigmas de programação, que é o paradigma orientado a objetos. Paradigmas de programação são um conjunto de técnicas que são utilizadas para formar a estrutura de um sistema. O principal paradigma usado nessa arquitetura é o orientado a objetos.

Este paradigma enfatiza a interação entre objetos, e o código é estruturado em torno de entidades do mundo real. Quando falamos que ele é modelado em volta de entidades do mundo real, significa que criamos classes que simulam coisas que realmente existem. Por exemplo, uma classe “Livro”, simula uma entidade do mundo real, que seria um livro de verdade. E, ao criar um objeto a partir dessa classe definida, ele passa a existir no nosso programa, no nosso sistema.

Este paradigma é comumente usado em linguagens como Java, que é uma linguagem orientada a objetos, e C#, que é uma linguagem de programação multiparadigma, incluindo o paradigma orientado a objetos.

Vantagens:

- Mapeamento natural para problemas do mundo real;
- Modularidade;
- Manutenibilidade; e
- Escalabilidade.

As vantagens de utilizarmos essa arquitetura incluem um mapeamento natural para problemas do mundo real. Ela simula entidades que existem e traz essas entidades para dentro do nosso código. Dessa forma, temos esse mapeamento de coisas que realmente existem dentro do nosso sistema, do nosso programa.

Outra vantagem é a modularidade, porque cada módulo, nesse caso, dentro da nossa arquitetura, é um objeto. Isso facilita a manutenção e a escalabilidade de um objeto. Adicionar uma funcionalidade nova ou alterar uma já existente é mais fácil, porque os objetos funcionam de forma independente. Em conjunto, eles integram o sistema, mas

foram projetados para funcionar de forma independente. Cada objeto tem sua função, o que facilita a manutenção do código e a escalabilidade, ou seja, a extensão do código com novas funcionalidades.

Ao contrário da arquitetura monolítica, em que todos os elementos eram interdependentes, na arquitetura orientada a objetos, cada objeto funciona de forma independente.

CLASSES

São um conjunto de instruções para construir objetos.

Define:

- Tipos de dados;
- Atributos; e
- Métodos.

Exemplo:

- Classe: Livro
- Atributos: título, autora, ano de publicação
- Métodos: emprestar e devolver

As classes são um conjunto de instruções para construir objetos. Podemos pensar em uma classe como se fosse um formulário. Quando você vai preencher um formulário, ele tem as categorias que você precisa preencher com suas informações pessoais, mas ele é só uma folha com as instruções. Não tem as características dessas informações, por exemplo, o nome ou o CPF. Ele vai estar vazio, mas vai indicar que você precisa preencher com o seu nome e o seu CPF.

A professora destaca que gosta de pensar nas classes como um formulário vazio, que define o tipo de dado que vai existir nos nossos objetos, os atributos e os métodos dentro do nosso objeto. Um exemplo de classe seria a classe “Livro”, que teria os atributos título, autor, ano de publicação e métodos como emprestar e devolver. Na formação dessa classe, apesar de ter esses atributos indicando o que precisa haver no livro para que ele seja considerado uma entidade real, essas informações estão vazias.

OBJETOS

São instâncias de classes – versão real do que a classe definiu.

Enquanto a classe define o que pode ser, o objeto é.

Exemplo: Objeto da classe Livro.

Atributos:

- Título: Jogos Vorazes
- Autora: Suzanne Collins
- Ano de Publicação: 2008



Métodos:

- Emprestar e devolver.

Os objetos, portanto, são instâncias de classes e representam a materialização do que foi definido pela classe. Ao passo que a classe estabelece o que um objeto pode ser, o objeto é a concretização dessa definição, existindo com dados reais no sistema.

ENCAPSULAMENTO

- Esconde os dados internos do objeto (atributos privados).
- Estes só podem ser acessados por meio de métodos controlados (métodos públicos).

Outro conceito importante na arquitetura orientada a objetos é o encapsulamento. O encapsulamento permite que certos dados do sistema sejam escondidos, garantindo que informações sensíveis não estejam acessíveis diretamente no código.

Para acessar esses dados, são criados métodos públicos que recuperam as informações de maneira segura, sem expô-las diretamente. Esses métodos servem como intermediários entre os dados privados e o restante do sistema, protegendo a integridade e a segurança das informações.

Além de proteger os dados, o encapsulamento também facilita a manutenção do código, pois as informações sensíveis estão isoladas e controladas por métodos específicos, reduzindo a possibilidade de alterações acidentais ou acesso não autorizado. Isso é especialmente importante em sistemas complexos, em que a segurança e a integridade dos dados são prioridades.

ATRIBUTOS PRIVADOS:

- não podem ser acessados fora da classe definida.

Dentro da arquitetura orientada a objetos, os atributos privados desempenham um papel importante na segurança e organização do código. Esses atributos são definidos dentro de uma classe e não podem ser acessados diretamente de fora dela. Por exemplo, ao criar uma classe "Pessoa", é possível definir atributos como nome, CPF, e-mail e senha. Destes, o CPF e a senha podem ser definidos como atributos privados, ou seja, não estarão acessíveis diretamente por outras partes do código.

CONSTRUTORES:

- usados para inicializar os atributos do novo objeto com valores específicos.

O uso de construtores é também uma prática comum ao inicializar esses atributos quando um novo objeto é criado. O construtor recebe os valores necessários para os atributos e os atribui ao objeto no momento de sua criação.

Por exemplo, ao criar um objeto a partir da classe “Pessoa”, o construtor pode receber parâmetros como nome, CPF, e-mail e senha, inicializando os atributos do objeto com esses valores. Dessa forma, é possível garantir que todos os atributos necessários estejam preenchidos corretamente ao criar o objeto.

MÉTODOS PÚBLICOS:

- Getters: permitem a leitura dos atributos privados.
- Setters: permitem modificar os valores de atributos privados.
- Métodos: simulam ações que podem ser realizadas.

Para acessar ou modificar esses atributos privados, utilizam métodos públicos, conhecidos como métodos de leitura (getters) e de modificação (setters). Esses métodos permitem controlar o acesso aos dados, garantindo que as informações sensíveis sejam manipuladas de maneira segura.

EXEMPLO (PSEUDO-CÓDIGO):

```
Classe Livro
  Atributos Privados
    titulo: String
    autora: String
    anoDePublicacao: Inteiro

  Construtor(titulo, autora, anoDePublicacao)
    this.titulo = titulo
    this.autora = autora
    this.anoDePublicacao = anoDePublicacao

  Método Público getTitulo()
    Retornar titulo

  Método Público setTitulo(novoTitulo)
    Se novoTitulo não é nulo e não é vazio então
      this.titulo = novoTitulo

  Método Público getAnoDePublicacao()
    Retornar anoDePublicacao

  Método Público emprestar()
    Imprimir "O livro " + titulo + " foi emprestado."

  Método Público devolver()
    Imprimir "O livro " + titulo + " foi devolvido."

Fim Classe
```



DIRETO DO CONCURSO

01. (CESPE/CEBRASPE/2024/CNPQ/ANALISTA EM CIÊNCIA E TECNOLOGIA PLENO I – ESPECIALIDADE: DESENVOLVIMENTO E ARQUITETURA DE SOFTWARE) Julgue o próximo item, no que se refere à arquitetura de sistemas.

Na arquitetura orientada a objeto, os dados e as operações são encapsulados para facilitar a manipulação das informações.



Vale lembrar que “encapsular” é o ato de “esconder” os dados por questões de segurança e para facilitar a manipulação das informações.

ABSTRAÇÃO

- Esconde a complexidade de um sistema de software.
- Permite a criação de modelos simples de entidades complexas do mundo real.
- Permite que a pessoa programadora se concentre em interações de nível mais alto, sem preocupação com detalhes.

Outro conceito fundamental na arquitetura orientada a objetos é a abstração. A abstração tem como objetivo esconder a complexidade do sistema de software, permitindo que o desenvolvedor se concentre nas interações de alto nível, sem a necessidade de lidar com os detalhes mais técnicos e específicos que ocorrem nos bastidores do sistema.

Em termos práticos, a abstração permite a criação de modelos simples para representar entidades complexas do mundo real. Isso é alcançado ao definir classes que encapsulam a lógica e os dados necessários para a operação do sistema, mas ocultam os detalhes de implementação que não são necessários para o uso cotidiano dessas classes. Dessa forma, o desenvolvedor pode trabalhar com essas entidades de forma simplificada, utilizando métodos e atributos que refletem diretamente as necessidades do sistema, sem se preocupar com a forma como esses métodos e atributos são implementados internamente.

Exemplo:

```
Classe Abstrata ItemBiblioteca
  Atributos Protegidos
    título: String
    autor: String
    anoDePublicação: Inteiro

  Método Abstrato emprestar()
  Método Abstrato devolver()
```

```
Classe Livro Extende ItemBiblioteca
    Método Público emprestar()
        Implementação específica para emprestar um livro
        Imprimir "Livro emprestado: " + título

    Método Público devolver()
        Implementação específica para devolver um livro
        Imprimir "Livro devolvido: " + título
```

```
Classe Revista Extende ItemBiblioteca
    Atributos Privados
        edição: Inteiro

    Método Público emprestar()
        Implementação específica para emprestar uma revista
        Imprimir "Revista emprestada: Edição " + edição

    Método Público devolver()
        Implementação específica para devolver uma revista
        Imprimir "Revista devolvida: Edição " + edição
```

Por exemplo, considere a classe “Item de Biblioteca”, que pode servir como uma classe base para outros tipos de itens, como livros ou revistas. A classe “Item de Biblioteca” pode definir atributos comuns a todos os itens, como título, autor e ano de publicação, além de métodos para emprestar e devolver o item. No entanto, as classes derivadas, como “Livro” e “Revista”, podem especificar comportamentos e atributos adicionais, utilizando a abstração para herdar as características gerais da classe base enquanto adicionam suas próprias especificidades.

Essa abordagem permite que diferentes tipos de itens de biblioteca sejam manipulados de maneira consistente, utilizando a mesma interface, mas com a flexibilidade de cada tipo de item ter suas próprias particularidades. Assim, a abstração facilita a manutenção e a extensão do sistema, permitindo que novas funcionalidades sejam adicionadas sem a necessidade de alterar a estrutura básica do código.

Outro exemplo de abstração pode ser visto na maneira como diferentes classes, como “Livro” e “Revista”, podem implementar o método de emprestar de maneira diferente, dependendo das necessidades específicas de cada tipo de item. O conceito de abstração é, portanto, essencial para reduzir a complexidade do código e permitir que o desenvolvedor se concentre nas funcionalidades principais do sistema, sem se preocupar com os detalhes de implementação subjacentes.



20m



DIRETO DO CONCURSO

02. (CESPE/CEBRASPE/2024/CNPQ/ANALISTA EM CIÊNCIA E TECNOLOGIA PLENO I/ ESPECIALIDADE: DESENVOLVIMENTO E ARQUITETURA DE SOFTWARE) Julgue o item subsecutivo, com relação a requisitos e componentes de software.

O design de software pode ser aplicado como produto, em uma abstração do mundo real, ou como processo, orientado aos objetivos do software.



A afirmação trazida pela banca é verdadeira ao afirmar que o design de software pode ser visto tanto como um produto, que é uma representação concreta de algo do mundo real, quanto como um processo, que orienta as práticas de desenvolvimento em direção aos objetivos do software.

HERANÇA

- Permite que uma classe herde atributos e métodos de outras classes;
- Isso permite a reutilização do código; e
- Estabelece uma relação hierárquica entre classes.

A herança é um dos pilares da programação e da arquitetura orientada a objetos. Ela define que uma classe pode herdar atributos e métodos de outra classe, permitindo a reutilização do código e estabelecendo uma relação hierárquica entre as classes.

Por exemplo, considera-se uma classe base chamada “Animal”, que contém atributos como nome e idade, além de métodos como mover e emitir som. A partir dessa classe, é possível derivar outras classes, como “Cachorro” e “Gato”, que herdam os atributos e métodos da classe base “Animal”. Além disso, essas classes derivadas podem incluir atributos e métodos específicos, como raça e latir no caso da classe “Cachorro”.

HERANÇA SIMPLES:

- subclasse herda apenas da superclasse (classe pai).

```
Classe Animal
  Atributos: nome, idade
  Métodos: mover, emitirSom

Classe Cão Extende Animal
  Atributos: raça
  Métodos: latir (sobrescreve emitirSom)
```




25m

HERANÇA MÚLTIPLA:

- classe derivada herda atributos de mais de uma classe base.

```
Classe Terrestre
  Métodos: caminhar

Classe Aquático
  Métodos: nadar

Classe Anfíbio Extende Terrestre, Aquático
  Métodos: caminhar (herdado de Terrestre), nadar (herdado de Aquático)
```

Ou seja, existem diferentes tipos de herança. A herança simples ocorre quando uma classe herda de uma única classe base. Já a herança múltipla, permitida em algumas linguagens de programação, permite que uma classe herde de mais de uma classe base, combinando os atributos e métodos de várias classes em uma única subclasse.

Por exemplo, uma classe “Anfíbio” poderia herdar características de classes “Terrestre” e “Aquático”, resultando em uma classe que combina as características de ambas. No entanto, a herança múltipla pode introduzir desafios, como o problema do diamante, em que uma subclasse herda os mesmos métodos de várias classes base, causando possíveis ambiguidades.

GABARITO

01. C

02. C

Este material foi elaborado pela equipe pedagógica do Gran Concursos, de acordo com a aula preparada e ministrada pela professora Ana Júlia Batista de Souza.

A presente gravação tem como objetivo auxiliar no acompanhamento e na revisão do conteúdo ministrado na videoaula. Não recomendamos a substituição do estudo em vídeo pela leitura exclusiva deste material.
